

BotXRGame — Session 2 Progress Log

First Successful Build on Samsung Galaxy XR Headset

Date: July 2026 | Author: swapnil-0

1. Session Overview

This session focused on getting the first working build of the BotXRGame app running on the Samsung Galaxy XR headset. Key work included fixing passthrough/MR configuration issues, importing and converting a 3D asset, cleaning up the scene, and successfully deploying to the headset.

Result: The app launched on the headset showing the Sci-Fi Space Fighter model in mixed reality. First successful on-device demo achieved.

2. Documentation Added to Repository

Two key documents were created and pushed to GitHub for team and stakeholder communication:

File	Purpose
docs/architecture/system-overview.md	Full system architecture, game flow, team roles, hardware/software stack, current state
docs/ros-interface/ros-unity-interface.md	All ROS 2 topics, message types, data directions, coordinate systems, setup instructions
docs/BotXRGame_Setup_Log.pdf	PDF log of session 1 setup work (GitHub, Unity, ADB)

```
git add .
git commit -m "Add system architecture and ROS-Unity interface documentation"
git push origin main
```

3. Scene Cleanup

The Mixed Reality template scene was cleaned up to remove tutorial and placeholder objects not relevant to BotXRGame. The following GameObjects were deleted from the Hierarchy:

- 01_Box_Yellow
- 02_Box_Orange
- 03_Box_Orange
- 04_Box_Red
- Tutorial
- Hidden_Area_1
- Hidden_Area_2
- Hidden_Area_3
- SpawnPoint_1

- SpawnPoint_2
- Postprocessing

Remaining scene objects after cleanup:

- ✓ Directional Light
 - ✓ Ground
 - ✓ Collectibles (with Collectible_Star x3)
 - ✓ Fighter03 (spaceship asset)
-

4. 3D Asset Import and URP Conversion

4.1 Asset Selected

Asset: Sci-Fi Space Fighter PBBRP by DUNGEON MASON

Source: Unity Asset Store (Free, Standard Unity Asset Store EULA, URP compatible)

Imported to: Assets/Sci-FiSpaceFighterPBR/ (Material, Mesh, Prefab, Scene, Texture folders)

4.2 URP Material Conversion

Despite being marked URP compatible, materials appeared pink (Built-in pipeline) after import. Fixed using the Render Pipeline Converter:

Window -> Rendering -> Render Pipeline Converter -> Built-in to URP -> Material Upgrade -> Initialize Converters -> Convert Assets

Target Pipeline selected: URP (Universal Render) — NOT 2D Render

4.3 Asset Placement and Scaling

Dragged Fighter03 prefab from Prefab folder into Scene Hierarchy. Scaled down to 0.25x using Transform component in Inspector (Scale X: 0.25, Y: 0.25, Z: 0.25).

5. XR and Passthrough Configuration Fixes

5.1 Problem: Black Screen on First Build

Initial build deployed to headset showed only a black rectangle. Root causes identified:

1. No XR Camera Rig in the scene (accidentally missing from template)
2. AR Camera Manager component not added to camera
3. Camera clear flags not set correctly for passthrough
4. OpenXR features not fully configured

5.2 Fix: Add XR Origin

Added XR camera rig via: GameObject -> XR -> XR Origin (VR)

Resulting hierarchy:

```
XR Origin (VR)
  Camera Offset
    Main Camera
```

5.3 Fix: Add AR Camera Manager

Selected Main Camera in Hierarchy -> Inspector -> Add Component -> AR Camera Manager. Enabled with default settings:

Auto Focus: enabled | Facing Direction: World | Render Mode: Any

5.4 Fix: Add AR Session

Added via: GameObject -> XR -> AR Session. Placed at bottom of Hierarchy.

5.5 Fix: Camera Clear Flags for Passthrough

Project Validation showed warning: 'Passthrough requires Camera clear flags set to solid color with alpha value zero.'

Applied fix using the Fix button in: Edit -> Project Settings -> XR Plug-in Management -> Project Validation

5.6 OpenXR Features Enabled

Edit -> Project Settings -> XR Plug-in Management -> OpenXR -> Android tab -> Features:

Feature	Status	Purpose
Android XR: AR Camera	Enabled	Passthrough camera feed
Android XR: AR Session	Enabled	Initializes AR/MR session
Android XR: AR Anchors	Enabled	Spatial anchors for game objects
Android XR: AR Planes	Enabled	Floor/surface detection
Hand Interaction Profile	Enabled	Hand tracking input
Hand Tracking Subsystem	Enabled	Required for hand mesh data

6. ADB Reconnection and Build Deployment

6.1 ADB Reconnection Process

Wireless ADB connection drops when headset sleeps — this is normal behavior. Reconnection process each session:

1. On headset: Settings -> Developer Options -> Wireless Debugging -> Pair with pairing code
2. Get new pairing port and 6-digit code from headset
3. Run pair command, then connect command

ADB path (Unity bundled):

```
cd "/c/Program Files/Unity/Hub/Editor/6000.4.2f1/Editor/Data/PlaybackEngines/AndroidPlaye
r/SDK/platform-tools"

./adb pair :
# Enter pairing code when prompted
./adb connect :
./adb devices
# Should show device listed
```

Note: Connection and pairing ports change each session. Always get fresh values from headset Wireless Debugging screen.

6.2 Build and Deploy

File -> Build Profiles -> Android XR -> Run Device: Default device -> Build and Run

APK saved to: BotXRGame/builds/BotXRGame_v0.1.apk

Bundle ID set in Build Profile Player Settings: com.swapnil.BotXRGame

6.3 Result

App launched successfully on Samsung Galaxy XR headset. Player could see:

- ✓ Real world passthrough (room, floor, surroundings)
- ✓ Sci-Fi Space Fighter model visible in MR space
- ✓ Stars from Collectibles group visible in scene

7. Known Warnings — Intentionally Ignored

Warning	Reason Ignored
[Optional] Switch to InputSystem.XR.PoseControl	Optional, no impact on project
[Optional] Switch to StickControl thumbsticks	Optional, no impact on project
Composition Layers Support feature required	Not using composition layers
OpenXR recommended version is 1.16.1 (have 1.17.1)	Newer version, fully compatible

8. Current Status

- ✓ GitHub repo created and connected (swapnil-0/BotXRGame)
- ✓ Unity 6000.4.2f1 MR project created and configured
- ✓ All packages installed (ROS TCP Connector, OpenXR, Android XR, XR Interaction Toolkit)
- ✓ Scene cleaned up — tutorial and placeholder objects removed
- ✓ Sci-Fi Space Fighter asset imported and URP converted
- ✓ XR Origin (VR) added with Main Camera
- ✓ AR Camera Manager added to Main Camera
- ✓ AR Session added to scene

- ✓ OpenXR features configured (AR Camera, AR Session, AR Anchors, AR Planes, Hand Tracking)
- ✓ Passthrough camera clear flags fixed
- ✓ Hand Interaction Profile added
- ✓ Android XR Build Profile active with correct Bundle ID
- ✓ Wireless ADB connection to Galaxy XR headset established
- ✓ First successful build deployed to Samsung Galaxy XR headset
- ✓ Spaceship visible in MR passthrough on headset
- ✓ System architecture and ROS interface docs pushed to GitHub

Next Steps

- Implement virtual joystick UI (hand tracking pinch gesture)
- Add bot movement C# script driven by joystick values
- Add HUD panel showing live cmd_vel ROS message values
- Add ROSConnection GameObject and configure ROS TCP bridge
- Position spaceship on ground plane correctly
- Build demo: joystick -> virtual bot movement -> ROS message display
- Install ROS 2 Humble on RubikPi (ROS Dev)
- Configure and test ros_tcp_endpoint (ROS Dev)
- Test full Unity to ROS communication over WiFi